gemm_vpu

Vector Processing Unit: bias, requantize, ReLU

Port list, module con, pipeline và timing chính xác theo chu kỳ

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Module	`gemm_vpu` (instance `u_vpu` trong `gemm_accel_top`)
Module con	`gemm_vpu_lane` (P instance), `gemm_vpu_requant` (trong mỗi lane)
Tuân theo	Spec hệ thống v1.0: mục 2.1 (FUN-02 … FUN-04, FUN-07), mục 4.3 (quyền truy cập SRAM), mục 5.4 (SPM-02, SPM-04), mục 5.6 (VPU-01 … VPU-05). Spec module `gemm_systolic_array` v1.0: mục 3 (luồng `o_row_*`, tham số `L_VPU`)
Kèm theo	`gemm_vpu_model.py`: golden bit-exact và mô hình cycle-accurate của đúng pipeline này; mọi con số trong tài liệu được kiểm bằng mô hình đó

Mục lục

1 Chức năng **3**

2 Tham số **3**

3 Port list **4**

3.1 Quy tắc giao tiếp 5

3.2 Ngữ nghĩa lệnh ghi C 5

4 Vi kiến trúc **6**

5 Module con **7**

5.1 `gemm_vpu_lane` 7

5.2 `gemm_vpu_requant` 8

5.3 Logic trong top 9

6 Timing **9**

6.1 Ví dụ: dạng sóng 9

7 Reset và soft clear **10**

8 Quy tắc hiện thực **11**

1 Chức năng

VPU nằm giữa đường xả của systolic array và cổng ghi buffer C. Mỗi chu kỳ nó nhận tối đa một đoạn hàng P giá trị INT32 từ array. Khi `POST_EN` = 1, nó cộng bias, requantize, ReLU và kẹp từng lane, rồi ghi Y (INT8) vào buffer C. Khi `POST_EN` = 0, nó ghi nguyên C (INT32). Độ trễ cố định `LAT` chu kỳ, thông lượng một đoạn hàng mỗi chu kỳ, không bao giờ chờ và không có đầu ra lỗi.

Bảng 1.1: Chức năng

ID	Yêu cầu
VPU-F01	Với mỗi đoạn hàng (i, j_0, mask) nhận được và mỗi lane c có mask = 1, j = $j_0 + c$: nếu `POST_EN` = 1, ghi $Y[i][j]$ theo công thức FUN-03 dưới đây vào buffer C ở bố cục INT8; nếu `POST_EN` = 0, ghi $C[i][j]$ không đổi ở bố cục INT32
VPU-F02	Lane có mask = 0 không được ghi. Mỗi đoạn hàng vào sinh đúng một lệnh ghi ra, đúng `LAT` chu kỳ sau
VPU-F03	Kết quả khớp từng bit với `post_ref()` của `gemm_vpu_model.py` (tức mô hình tham chiếu mục 2.4 của spec hệ thống) với mọi giá trị đầu vào INT32, mọi bias INT32, mọi M_0 INT32, mọi sh 0 … 31
VPU-F04	VPU chỉ đọc buffer BIAS và chỉ ghi buffer C; không đọc buffer C, không đọc A, B, không truy cập DRAM, không giữ trạng thái giữa các tác vụ
VPU-F05	Kết quả không phụ thuộc P và không phụ thuộc `MUL_STAGES`

$$
\begin{aligned}
acc &= C[i][j] + bias[j] && \text{(INT32, wrap mod } 2^{32}\text{)} \\
prod &= acc \cdot M_0 && \text{(tích có dấu 64 bit, không bao giờ tràn)} \\
y &= (prod + 2^{30+sh}) \gg (31 + sh) && \text{(dịch số học: làm tròn nửa lên)} \\
y &= \max(y, 0) \text{ nếu } \texttt{RELU_EN} = 1 \\
Y[i][j] &= \min(\max(y, -128), 127) && \text{(INT8)}
\end{aligned}
$$

Ý nghĩa của từng bước (bias là gì, vì sao nhân với M_0 rồi dịch) ở tài liệu "Thuật toán GEMM, systolic array và VPU", mục 2, 3 và 6.

2 Tham số

Bảng 2.1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Số lane; chỉ nhận 4 hoặc 8. Nối từ `PE_ARRAY_SIZE` của top, cùng giá trị với array
`MUL_STAGES`	1	Số tầng của bộ nhân $acc \cdot M_0$: 1 hoặc 2. Chọn 2 khi bộ nhân 32 × 32 một chu kỳ không đạt tần số mục tiêu
`LAT`	4	localparam = 3 + `MUL_STAGES`: độ trễ từ `i_row_valid` tới `o_c_wr_en`
`IDX_W`	5	localparam: độ rộng chỉ số hàng, cột (0 … 31)

Bảng 2.2: Tham số

ID	Yêu cầu
VPU-P01	P ∉ {4, 8} hoặc `MUL_STAGES` ∉ {1, 2} phải bị chặn lúc elaboration (`$error` trong `generate`)
VPU-P02	Giá trị `MUL_STAGES` chỉ khai báo một lần, trong package `gemm_pkg`: `VPU_MUL_STAGES` và `VPU_LATENCY = 3 + VPU_MUL_STAGES`. `gemm_accel_top` nối `MUL_STAGES = gemm_pkg::VPU_MUL_STAGES` cho VPU và `L_VPU = gemm_pkg::VPU_LATENCY` cho array. Không được viết số 4 hay 5 trực tiếp ở top
VPU-P03	`gemm_vpu` có kiểm tra elaboration: `LAT == gemm_pkg::VPU_LATENCY`; sai thì `$error`

3 Port list

Bảng 3.1: Port của `gemm_vpu`

Port	Rộng	Chiều	Nối với	Mô tả
Clock, reset				
`clk`	1	In	top	Clock
`rst_n`	1	In	`rst_n_sync`	Reset bất đồng bộ tích cực thấp, đã đồng bộ sườn nhả
`i_soft_clr`	1	In	FSM	Xung xóa đồng bộ 1 chu kỳ (`soft_clr`)
Cấu hình của job đang tính (từ FSM, `o_comp_*`; ổn định suốt COMPUTE)				
`i_post_en`	1	In	FSM	`POST_EN` của job
`i_relu_en`	1	In	FSM	`RELU_EN` của job; bị bỏ qua khi `i_post_en` = 0
`i_quant_m0`	32	In	FSM	M_0 của job, INT32 có dấu
`i_quant_shift`	5	In	FSM	sh của job, không dấu
Luồng hàng vào, từ array (không có ready)				
`i_row_valid`	1	In	Array	Có một đoạn hàng trong chu kỳ này
`i_row_idx`	5	In	Array	Hàng i
`i_row_col`	5	In	Array	Cột đầu j_0 (bội của P)
`i_row_data`	32P	In	Array	Lane c (bit [32c+31 : 32c]) = $C[i][j_0 + c]$, INT32
`i_row_mask`	P	In	Array	Bit c = 1: lane c hợp lệ
Cổng đọc buffer BIAS (VPU là master, không có ready)				
`o_bias_rd_en`	1	Out	SRAM	Yêu cầu đọc một vector bias
`o_bias_rd_col`	5	Out	SRAM	j_0 (bội của P)
`i_bias_rd_data`	32P	In	SRAM	Lane c (bit [32c+31 : 32c]) = $bias[j_0 + c]$; hợp lệ 1 chu kỳ sau yêu cầu
Cổng ghi buffer C (VPU là master, không có ready)				
`o_c_wr_en`	1	Out	SRAM	Lệnh ghi một đoạn hàng, có hiệu lực ở cạnh clock cuối chu kỳ này
`o_c_wr_fmt`	1	Out	SRAM	0: bố cục INT32; 1: bố cục INT8
`o_c_wr_row`	5	Out	SRAM	Hàng i
`o_c_wr_col`	5	Out	SRAM	Cột đầu j_0 (bội của P)
`o_c_wr_data`	32P	Out	SRAM	INT32: lane c ở bit [32c+31 : 32c]. INT8: lane c ở bit [8c+7 : 8c], bit [32P−1 : 8P] = 0
`o_c_wr_mask`	P	Out	SRAM	Bit c = 1: ghi lane c; bằng `i_row_mask` của đoạn hàng tương ứng

3.1 Quy tắc giao tiếp

Array → VPU

TRAO ĐỔI GÌ?
Một đoạn hàng P giá trị INT32 kèm hàng, cột đầu và mask.

AI KHỞI TẠO?
Array đẩy; VPU nhận. Không có ready.

KHI NÀO ĐƯỢC CHẤP NHẬN?
VPU phải nhận đoạn hàng ở đúng chu kỳ `i_row_valid` = 1, kể cả khi các chu kỳ liền nhau đều có đoạn hàng.

KHI NÀO SANG BƯỚC TIẾP?
VPU ghi đoạn hàng đó vào buffer C đúng `LAT` chu kỳ sau. Array tính `o_pe_done` dựa trên con số này (`L_VPU` = `LAT`).

VPU ↔ SRAM (BIAS đọc, C ghi)

TRAO ĐỔI GÌ?
Đọc P word bias liền nhau; ghi một đoạn hàng của C, có mask.

AI KHỞI TẠO?
VPU là master trên cả hai cổng. SRAM không có backpressure.

KHI NÀO ĐƯỢC CHẤP NHẬN?
Đọc: dữ liệu về đúng 1 chu kỳ sau yêu cầu. Ghi: có hiệu lực ở cạnh clock cuối chu kỳ `o_c_wr_en` = 1.

KHI NÀO SANG BƯỚC TIẾP?
Chỉ dùng trong pha COMPUTE (bảng 4-2 của spec hệ thống): VPU là bên duy nhất đọc BIAS và ghi C.

Bảng 3.2: Giao tiếp

ID	Yêu cầu
VPU-I01	`i_row_*` được giả định đúng SA-11, SA-12 của spec array: lấy thẳng từ thanh ghi; `i_row_col` là bội của P; mask có dạng các bit 1 liền nhau từ bit 0 (`i_row_mask[c]` = 1 khi và chỉ khi $j_0 + c < N$), khác 0
VPU-I02	`o_bias_rd_en` = `i_row_valid` ∧ `i_post_en`; `o_bias_rd_col` = `i_row_col`. Đây là đường tổ hợp duy nhất từ input tới output của VPU, được phép vì `i_row_*` lấy từ thanh ghi của array. Khi `i_post_en` = 0, VPU không bao giờ đọc BIAS (buffer BIAS không được nạp ở tác vụ đó)
VPU-I03	VPU chỉ dùng `i_bias_rd_data` ở chu kỳ ngay sau chu kỳ có `o_bias_rd_en` = 1. Lane có mask = 0 được phép mang giá trị bất kỳ, kể cả X trong mô phỏng; X không được lan sang lane có mask = 1
VPU-I04	Mọi port `o_c_wr_*` lấy từ thanh ghi. Khi `o_c_wr_en` = 1: `o_c_wr_fmt` = `i_post_en`; `o_c_wr_mask` ≠ 0. Khi `o_c_wr_en` = 0, các port khác có giá trị tùy ý
VPU-I05	Cấu hình `i_post_en`, `i_relu_en`, `i_quant_m0`, `i_quant_shift` ổn định từ `pe_start` tới `pe_done` (FSM-I03 của spec FSM). Vì pipeline rỗng trước đoạn hàng đầu và sau lệnh ghi cuối của mỗi tác vụ, VPU dùng cấu hình trực tiếp, không cần chốt hay đưa cấu hình theo pipeline

3.2 Ngữ nghĩa lệnh ghi C

Spec module SRAM phải hiện thực đúng bảng dưới (bố cục SPM-02 của spec hệ thống). Buffer C cần ghi theo byte.

Bảng 3.3: Mỗi lane c có `o_c_wr_mask[c]` = 1, j = j0 + c

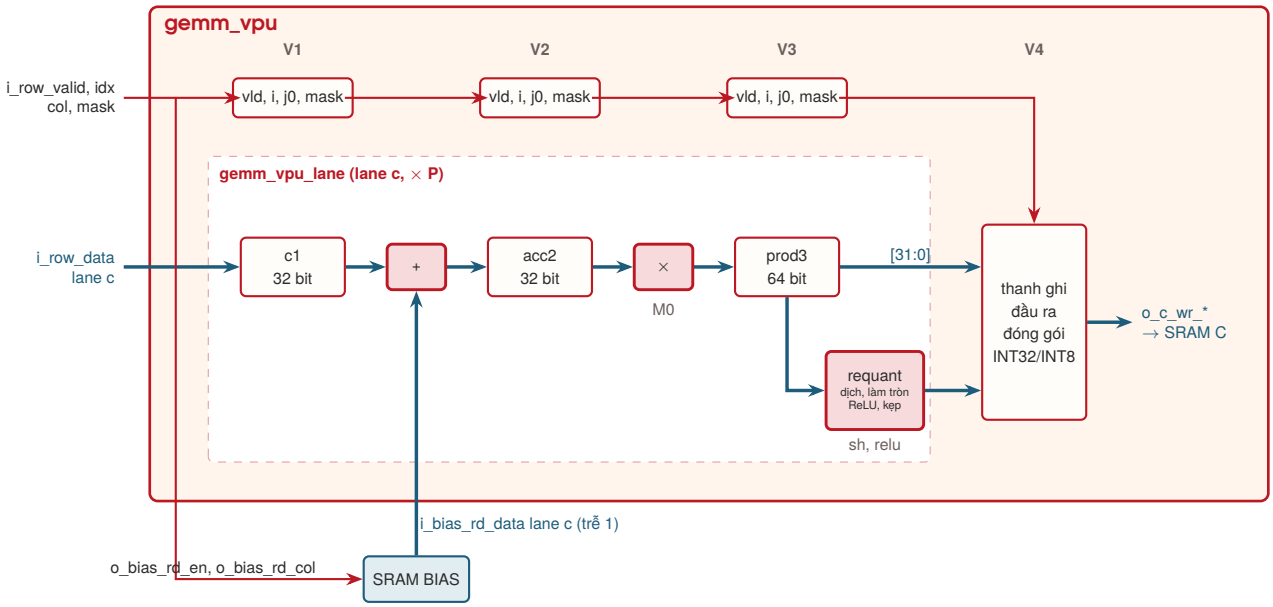
`o_c_wr_fmt`	Word được ghi	Nội dung
0 (INT32)	{i[4:0], j[4:0]}	Cả word ← `o_c_wr_data[32c+31 : 32c]`
1 (INT8)	{i[4:0], j[4:2]}	Byte lane j[1:0] ← `o_c_wr_data[8c+7 : 8c]`; ba byte còn lại của word giữ nguyên

Với P = 4, một đoạn hàng INT8 là đúng một word; với P = 8 là hai word liền nhau {i, $j_0/4$} và {i, $j_0/4 + 1$}. Vì N

là bội của 4, mask ở chế độ INT8 luôn phủ trọn word, nhưng SRAM vẫn phải dùng byte enable theo mask để đúng với mọi trường hợp.

4 Vi kiến trúc

VPU là pipeline thẳng `LAT` = 3 + `MUL_STAGES` tầng, không có trạng thái điều khiển: một chuỗi bit hợp lệ và metadata (hàng, cột, mask) chạy song song với P lane dữ liệu giống hệt nhau. Mỗi lane là một `gemm_vpu_lane`. Chế độ `POST_EN` = 0 đi qua cùng các tầng, nên độ trễ không đổi theo chế độ.



Hình 4.1: Pipeline của `gemm_vpu` với `MUL_STAGES` = 1. Mux theo `i_post_en` ở bộ cộng, bộ nhân và đầu ra không vẽ

Bảng 4.1: Các tầng pipeline (D = chu kỳ có `i_row_valid` = 1)

Tầng	Thanh ghi	Tính trong chu kỳ trước khi chốt	Có giá trị ở chu kỳ
V1	`c1`, meta1	Chốt `i_row_data`, `i_row_idx/col/mask`; phát đọc bias	D + 1
V2	`acc2`, meta2	`acc2 = post ? c1 + bias : c1` (32 bit wrap)	D + 2
V3	`prod3`, meta3	`prod3 = post ? acc2 × M0 : sext(acc2)` (64 bit), qua `MUL_STAGES` tầng	D + 2 + `MUL_STAGES`
V4	`o_c_wr_*`	requant(`prod3`) hoặc `prod3[31:0]`; đóng gói theo `fmt`	D + `LAT`

Bảng 4.2: Module con

Module	Instance	Vai trò
`gemm_vpu_lane`	`u_lane[c]`	Đường dữ liệu một lane: thanh ghi `c1`, `acc2`, `prod3`; bộ cộng bias; bộ nhân 32 × 32; đầu ra tổ hợp `o_y`
`gemm_vpu_requant`	`u_rq` (trong lane)	Tổ hợp: dịch có làm tròn, ReLU, kẹp; 64 bit → INT8
(logic trong top)	—	Chuỗi bit hợp lệ và metadata V1 … V3; cổng đọc bias; thanh ghi đầu ra V4 và đóng gói

5 Module con

5.1 gemm_vpu_lane

Bảng 5.1: Port của gemm_vpu_lane

Port	Rộng	Chiều	Mô tả
clk	1	In	Clock (lane không có reset, xem RST-03)
i_post_en, i_relu_en	1, 1	In	Cấu hình
i_m0	32	In	M_0, có dấu
i_sh	5	In	sh
i_c	32	In	Lane c của i_row_data (vào tầng V1)
i_bias	32	In	Lane c của i_bias_rd_data (vào tầng V2, cùng chu kỳ với c1)
o_y	32	Out	Tổ hợp từ prod3: post ? sext$_{32}$(Y) : prod3[31:0]

Bảng 5.2: gemm_vpu_lane

ID	Yêu cầu
LAN-01	c1 <= i_c mỗi chu kỳ. acc2 <= i_post_en ? c1 + i_bias : c1, cộng 32 bit wrap. Mux phải chọn c1 (không phải cộng với 0) để X của i_bias không lan khi không đọc bias
LAN-02	prod3 = acc2 × i_m0, nhân có dấu 32 × 32 → 64 bit, khi i_post_en = 1; = acc2 mở rộng dấu lên 64 bit khi i_post_en = 0. Mux có thể đặt ở bất kỳ tầng nào trong bộ nhân vì cấu hình không đổi trong tác vụ
LAN-03	MUL_STAGES = 1: prod3 là một thanh ghi sau bộ nhân tổ hợp. MUL_STAGES = 2: thêm đúng một tầng thanh ghi bên trong bộ nhân; cách chia do người hiện thực chọn (gợi ý ở notebox dưới). Lane có đúng MUL_STAGES + 2 tầng thanh ghi từ i_c tới prod3
LAN-04	o_y = i_post_en ? {{24{Y[7]}}, Y} : prod3[31:0], với Y = o_y của gemm_vpu_requant(prod3, i_sh, i_relu_en). Không có thanh ghi sau prod3 trong lane
LAN-05	Thanh ghi dữ liệu của lane cập nhật mọi chu kỳ (cho phép thêm enable theo bit hợp lệ để giảm công suất; kết quả không được thay đổi)

GỢI Ý CHIA BỘ NHÂN KHI MUL_STAGES = 2

Tách $M_0 = M_{hi} \cdot 2^{16} + M_{lo}$, với M_{hi} = M0[31:16] có dấu và M_{lo} = M0[15:0] không dấu. Tầng thứ nhất chốt hai tích riêng $p_{hi} = acc \cdot M_{hi}$ (48 bit có dấu) và $p_{lo} = acc \cdot M_{lo}$ (49 bit có dấu); tầng thứ hai chốt $prod = (p_{hi} \ll 16) + p_{lo}$. Mỗi tích riêng khớp tốt với 2 khối DSP của FPGA.

```
// Tham khảo (SystemVerilog, MUL_STAGES = 1)
logic signed [31:0] c1, acc2;
logic signed [63:0] prod3;
logic signed [7:0]  y8;
always_ff @(posedge clk) begin
  c1    <= i_c;
  acc2  <= i_post_en ? (c1 + i_bias) : c1;
  prod3 <= i_post_en ? (64'(acc2) * 64'(i_m0)) : 64'(acc2);
end
gemm_vpu_requant u_rq (.i_prod(prod3), .i_sh(i_sh), .i_relu(i_relu_en), .o_y(y8));
assign o_y = i_post_en ? 32'(y8) : prod3[31:0];   // 32'(y8): mở rộng dấu
```

5.2 gemm_vpu_requant

Bảng 5.3: Port của gemm_vpu_requant

Port	Rộng	Chiều	Mô tả
i_prod	64	In	$prod$, có dấu
i_sh	5	In	sh
i_relu	1	In	RELU_EN
o_y	8	Out	Y, INT8 có dấu. Module thuần tổ hợp

Công thức chuẩn $y = (prod + 2^{30+sh}) \gg (31 + sh)$ cần cộng 64 bit rồi dịch 64 bit. Module hiện thực một dạng tương đương rẻ hơn:

$$t = prod \ggg (30 + sh) \quad \text{(34 bit)}, \qquad y = (t + 1) \ggg 1 \quad \text{(33 bit)}$$

$$Y = \begin{cases} 127 & y > 127 \\ lo & y < lo \\ y[7:0] & \text{còn lại} \end{cases}, \qquad lo = \begin{cases} 0 & relu = 1 \\ -128 & relu = 0 \end{cases}$$

VÌ SAO HAI CÁCH CHO CÙNG KẾT QUẢ

Đặt $n = 31 + sh$ và viết $prod = t \cdot 2^{n-1} + r$ với $0 \le r < 2^{n-1}$ (tức $t = prod \ggg (n-1)$). Khi đó $(prod + 2^{n-1})/2^n = (t+1)/2 + r/2^n$, với $0 \le r/2^n < 1/2$. Nếu $t+1$ chẵn, phần nguyên là $(t+1)/2$; nếu $t+1$ lẻ, $(t+1)/2 = q + 1/2$ và cộng thêm một số nhỏ hơn 1/2 vẫn cho phần nguyên q. Cả hai trường hợp đều bằng $(t+1) \ggg 1$. ReLU rồi kẹp tương đương với kẹp vào $[lo, 127]$. Mô hình kiểm tra 203 136 trường hợp, gồm mọi tổ hợp biên và các giá trị đúng nửa.

Bảng 5.4: gemm_vpu_requant

ID	Yêu cầu
REQ-01	Kết quả phải bằng công thức chuẩn ở mục 1 với mọi i_prod trong $[-2^{62} + 2^{31}, 2^{62}]$ (miền của $acc \cdot M_0$), mọi i_sh, mọi i_relu. Được dùng dạng hiện thực bất kỳ thỏa điều này
REQ-02	Độ rộng tối thiểu: $\lvert prod \rvert \le 2^{62}$ và dịch ≥ 30 nên $t \in [-2^{32}, 2^{32}]$, cần 34 bit có dấu; y cần 33 bit có dấu. Không được cắt ngắn hơn: cắt t về 32 bit sai ở $acc = M_0 = -2^{31}$
REQ-03	Lượng dịch $30 + sh \in$ [30, 61]. Bộ dịch chỉ cần sinh 34 bit của t, tức cửa sổ i_prod[63 : 30+sh] mở rộng dấu lên 34 bit; hiện thực được bằng mux 32 vị trí, không cần bộ dịch 64 bit đầy đủ
REQ-04	Dịch là dịch số học (giữ dấu); làm tròn là nửa lên (về phía +∞), không phải nửa ra xa 0

```
// Tham khảo (SystemVerilog)
logic signed [33:0] t;
logic signed [32:0] y, lo;
assign t  = 34'(i_prod >>> (6'd30 + 6'(i_sh)));
assign y  = 33'((35'(t) + 35'sd1) >>> 1);
assign lo = i_relu ? 33'sd0 : -33'sd128;
always_comb
  if      (y > 33'sd127) o_y = 8'sd127;
  else if (y < lo)       o_y = 8'(lo);
  else                   o_y = 8'(y);
```

5.3 Logic trong top

Bảng 5.5: Logic trong `gemm_vpu`

ID	Yêu cầu
TOP-01	Chuỗi hợp lệ: `vld1 <= i_row_valid`, `vld2 <= vld1`, rồi `MUL_STAGES` tầng tới `vld3`, `o_c_wr_en <= vld3`. Metadata (idx, col, mask) đi cùng chuỗi đó. Mọi bit hợp lệ reset và clear về 0; metadata không bắt buộc reset
TOP-02	Cổng đọc bias theo VPU-I02. Lane c nhận `i_c = i_row_data[32c +: 32]`, `i_bias = i_bias_rd_data[32c +: 32]`
TOP-03	Thanh ghi đầu ra V4, nạp mọi chu kỳ: `o_c_wr_fmt <= i_post_en`; `o_c_wr_row`, `o_c_wr_col`, `o_c_wr_mask <=` metadata của `vld3`. Dữ liệu: khi `i_post_en` = 0, `o_c_wr_data[32c +: 32] <= o_y` của lane c; khi `i_post_en` = 1, `o_c_wr_data[8c +: 8] <= o_y[7:0]` của lane c và `o_c_wr_data[32P-1 : 8P] <= 0`
TOP-04	Không có bộ đếm, FSM, hay trạng thái nào khác. VPU không biết M, N, K và không biết khi nào tác vụ kết thúc

6 Timing

Mọi lệnh ghi xảy ra đúng `LAT` chu kỳ sau đoạn hàng tương ứng. Thông lượng là một đoạn hàng mỗi chu kỳ với mọi mẫu đến (liền nhau hoặc có khoảng trống). Các công thức dưới đây là quy định; RTL phải khớp đúng từng chu kỳ.

Bảng 6.1: Thời điểm các sự kiện (D = chu kỳ có `i_row_valid` = 1)

Sự kiện	Chu kỳ
`o_bias_rd_en` (nếu `POST_EN` = 1)	D
`i_bias_rd_data` hợp lệ, `c1` hợp lệ	D + 1
`acc2` hợp lệ	D + 2
`prod3` hợp lệ	D + 2 + `MUL_STAGES`
`o_c_wr_en` = 1, dữ liệu ghi vào C ở cạnh clock cuối chu kỳ	D + `LAT` = D + 3 + `MUL_STAGES`
`o_pe_done` của array (đoạn hàng cuối ở D_{last})	D_{last} + `LAT` + 1

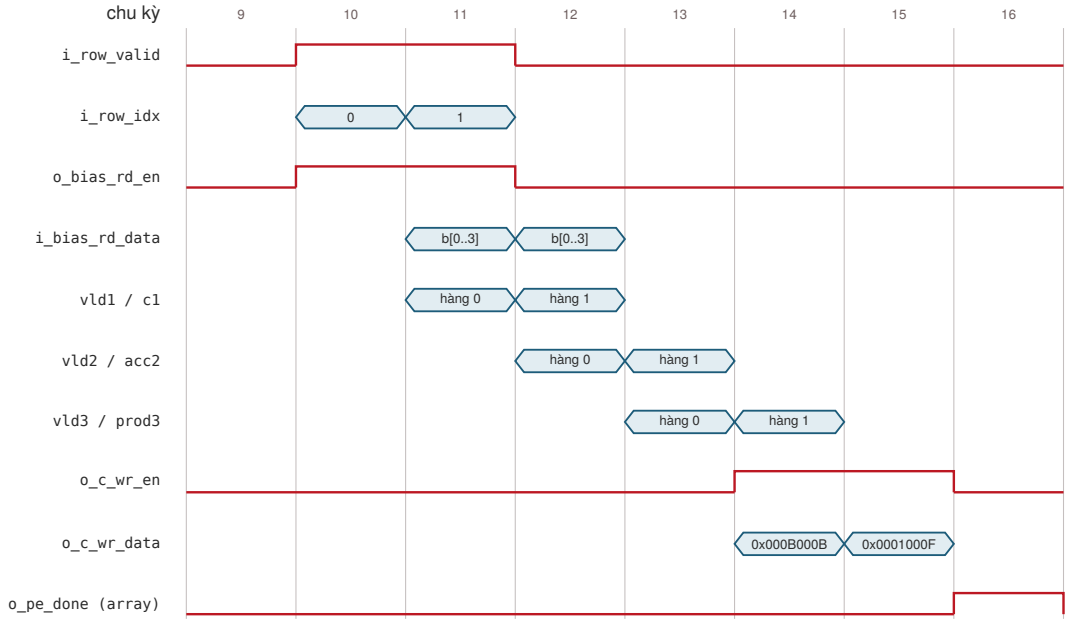
Ghép với spec array (bảng 6-1 của spec đó), số chu kỳ của cả pha COMPUTE là $(N_T - 1)\max(K, P) + K + P + r_{last} + \mathrm{LAT} + 3$:

Bảng 6.2: Số chu kỳ từ `pe_start` tới `pe_done` khi có VPU

(M, N, K)	P = 4, LAT = 4	P = 4, LAT = 5	P = 8, LAT = 4	P = 8, LAT = 5
(32, 32, 32)	2062	2063	534	535
(20, 12, 8)	134	135	66	67
(2, 4, 4) (ví dụ)	16	17	20	21

6.1 Ví dụ: dạng sóng

Ví dụ của tài liệu thuật toán với P = 4, `MUL_STAGES` = 1: M = 2, N = 4, K = 4, một tile. Array xả hàng 0 ở chu kỳ 10 và hàng 1 ở chu kỳ 11. Cấu hình `POST_EN` = 1, `RELU_EN` = 1, M_0 = 1 717 986 918, sh = 5, $bias = (500, -300, 200, -1000)$.



Hình 6.1: Dạng sóng của ví dụ, chu kỳ 9–16: hàng vào ở chu kỳ 10, 11; lệnh ghi ở chu kỳ 14, 15 (LAT = 4); `o_pe_done` ở chu kỳ 16

Bảng 6.3: Giá trị từng tầng của ví dụ

(i, j)	c1	bias	acc2	prod3	t = prod ⋙ 35	y	Y
(0,0)	−58	500	442	759 350 217 756	22	11	11
(0,1)	219	−300	−81	−139 156 940 358	−5	−2	0
(0,2)	250	200	450	773 094 113 100	22	11	11
(0,3)	7	−1000	−993	−1 705 961 009 574	−50	−25	0
(1,0)	92	500	592	1 017 048 255 456	29	15	15
(1,1)	93	−300	−207	−355 623 292 026	−11	−5	0
(1,2)	−173	200	27	46 385 646 786	1	1	1
(1,3)	−14	−1000	−1014	−1 742 038 734 852	−51	−25	0

Lệnh ghi ở chu kỳ 14 có `o_c_wr_fmt` = 1, hàng 0, cột 0, mask 4'b1111, dữ liệu byte 0 … 3 = (11, 0, 11, 0), tức word {0, 0} của C = `0x000B000B`. Chu kỳ 15 ghi word {1, 0} = `0x0001000F`. Với `POST_EN` = 0, cùng hai chu kỳ đó ghi bốn word INT32 mỗi hàng: (−58, 219, 250, 7) và (92, 93, −173, −14), và không có `o_bias_rd_en`.

7 Reset và soft clear

Bảng 7.1: Reset

ID	Yêu cầu
RST-01	`rst_n` = 0 (bất đồng bộ): `vld1`, `vld2`, các bit hợp lệ trong tầng nhân, `o_c_wr_en` về 0
RST-02	`i_soft_clr` = 1 (đồng bộ): tác động như RST-01 ở cạnh clock cuối chu kỳ đó; mọi đoạn hàng đang trong pipeline bị bỏ. Nếu `o_c_wr_en` đã bằng 1 trong chính chu kỳ có `i_soft_clr`, lệnh ghi đó vẫn xảy ra; điều này được phép vì nội dung scratchpad sau soft reset là không xác định (RST-04 spec hệ thống)
RST-03	Thanh ghi dữ liệu (`c1`, `acc2`, `prod3`, metadata, `o_c_wr_fmt/row/col/data/mask`) không bắt buộc reset; mọi đường dùng chúng đều bị chặn bởi bit hợp lệ

8 Quy tắc hiện thực

Bảng 8.1: Hiện thực

ID	Yêu cầu
IMP-01	SystemVerilog tổng hợp được; `always_ff @(posedge clk or negedge rst_n)` cho thanh ghi có reset, `always_ff @(posedge clk)` cho thanh ghi dữ liệu; không có latch
IMP-02	Mọi phép cộng, nhân, dịch, so sánh dùng kiểu `signed` rõ ràng; dịch phải dùng `>>>` trên toán hạng có dấu. Không dùng phép chia hay modulo
IMP-03	Mỗi module con trong file riêng, cùng tên module; P lane dựng bằng `generate`
IMP-04	Ngoài VPU-I02, không có đường tổ hợp từ input tới output của `gemm_vpu`
IMP-05	Đường dài nhất dự kiến là bộ nhân 32 × 32 (tầng V3) và chuỗi dịch, cộng 34 bit, so sánh (tầng V4). Nếu V4 không đạt tần số, được phép chuyển phép dịch sang cuối V3 (chốt t thay vì `prod3`) mà không đổi `LAT`

Bảng 8.2: Assertion bắt buộc trong RTL (SVA, chỉ trong mô phỏng)

ID	Yêu cầu
AS-01	`i_row_valid` → `i_row_col[$clog2(P)-1:0] == 0`
AS-02	`i_row_valid` → (`i_row_mask != 0`) ∧ `$onehot(i_row_mask + 1'b1)` (mask là dãy bit 1 liền nhau từ bit 0, hoặc toàn 1)
AS-03	`o_bias_rd_en` → `i_post_en`
AS-04	(`vld1` ∨ `vld2` ∨ … ∨ `o_c_wr_en`) → `$stable` của cả bốn tín hiệu cấu hình
AS-05	`o_c_wr_en` → `o_c_wr_fmt == i_post_en`
AS-06	`o_c_wr_en` ↔ `i_row_valid` trễ đúng `LAT` chu kỳ (khi không có `i_soft_clr` trong khoảng đó)
AS-07	Trong `gemm_accel_top`: `u_array.o_pe_done` → không còn bit hợp lệ nào trong pipeline VPU

Bảng 8.3: Tài nguyên ước tính (`MUL_STAGES` = 1)

Hạng mục	P = 4	P = 8
Bộ nhân 32 × 32 có dấu → 64 bit	4	8
Bộ cộng 32 bit (bias)	4	8
Bộ requant (dịch 32 vị trí × 34 bit, cộng 34 bit, 2 so sánh)	4	8
Thanh ghi dữ liệu lane (32 + 32 + 64 bit mỗi lane)	512 bit	1 024 bit
Thanh ghi đầu ra và metadata	khoảng 190 bit	khoảng 330 bit
DSP trên FPGA (khoảng 4 khối mỗi bộ nhân 32 × 32)	khoảng 16	khoảng 32